LangGraph notes

add_messages

how LangGraph remembers a conversation

(Annotated + List[BaseMessage] + add_messages, explained slowly)

Annotated
attaches a note to a type

+

List[BaseMessage]
what the field stores

+

add_messages
how updates get merged in

↓

= a chatbot that never forgets!



Inside: p2 the one line | p3 Annotated | p4 List[BaseMessage] | p5 add_messages | p6 a message's journey | p7 cheat sheet

Python | LangChain | LangGraph

The one line that does it all

(the line we are decoding today)

python

```python
from typing import Annotated, List, TypedDict
from langchain_core.messages import BaseMessage
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[List[BaseMessage], add_messages]
```

Zoom in on the highlighted line. It is built from **three** pieces:

`messages: Annotated[List[BaseMessage], add_messages]`

Annotated
Attaches **extra info** (metadata) to a type.

List[BaseMessage]
WHAT the field stores: a list of message objects.

add_messages
HOW to merge incoming messages with the existing ones.

Think of it this way

The type tells LangGraph **what** the data is. Annotated adds **how** LangGraph should handle that data.

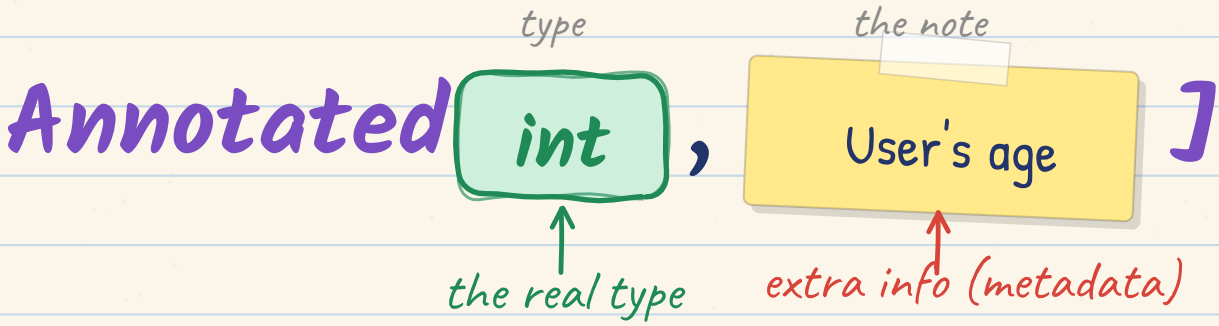
Annotated itself does NOT merge anything.

It just carries the note. add_messages does the merging!

Part 1: Annotated

a sticky note you stick onto a type

Annotated lets you attach ***extra information*** to a type. It is a plain Python typing feature - it is not specific to LangGraph.



python

```python
from typing import Annotated

Age = Annotated[int, "User's age"]
age: Age = 20
```

int is still int!
(age is just 20)

Python can ***expose*** that metadata to tools that look for it, but it does not change how the value behaves. `age` still acts like a normal `int`.

So what does LangGraph do with it?

add_messages

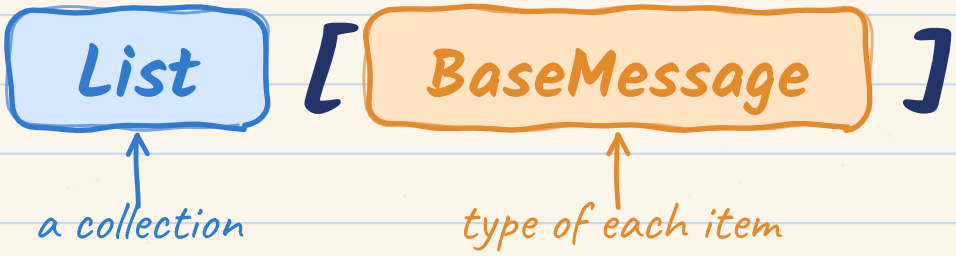
List[BaseMessage] → LangGraph reads the note and thinks: ***"ah - when a node updates this field, merge it using that reducer!"***

Annotated only carries the note. It never merges anything.

Part 2: List[BaseMessage]

what the field stores

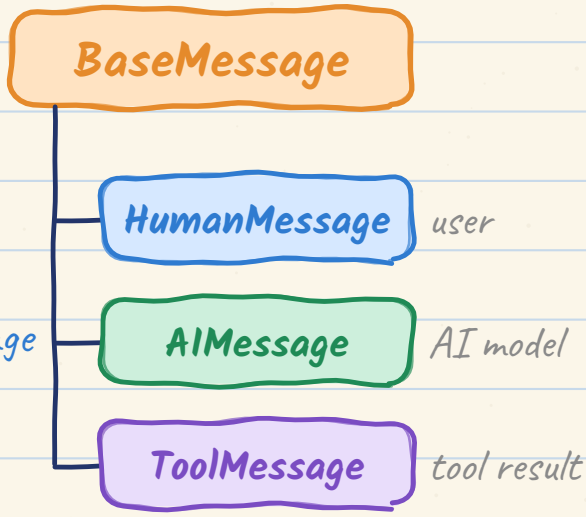
`List` means a collection that can hold many items. `BaseMessage` is the expected type of each item. Together: ***a list of message objects, in conversation order.***



a conversation, stored in order



one shared parent



Why BaseMessage and not HumanMessage?

`List[HumanMessage]` would describe a list of ***human*** messages only. But a conversation also holds AI replies and maybe tool results. `BaseMessage` is the shared parent type, so one list can hold them all.

Heads-up!

Type hints are not enforced at runtime. A type checker flags `["Hello"]`, but Python itself will not stop it.

Modern Python

You can also write `list[BaseMessage]` (lowercase). Same idea, available in Python 3.9+.

Part 3: add_messages

the reducer that remembers

A **reducer function** that decides how messages are ***added to*** or ***updated in*** your conversation state.

WHAT

A reducer function that manages how messages are added or updated in your state.

WHY

To keep the conversation history instead of replacing it each time a node returns a message.

WHEN

When building conversational agents that keep history across graph nodes and turns.

HOW

Attach it to the *messages* field using *Annotated*.

Same update, two different outcomes

Without add_messages

state is Hello Hi!

node returns How are you?

↓

result How are you? ✗

old history lost!

With add_messages

state is Hello Hi!

node returns How are you?

↓

result Hello Hi! How are you?

✓ *history kept!*

Smart with message IDs

If an incoming message has the ***same ID*** as an existing one, add_messages can ***update*** it instead of adding a duplicate.

id=2 "Hi!"

↓ *same id = updated!*

id=2 "Hi there!"

A message's journey

following "Hello" through the graph

1. **You send a message**
   `HumanMessage(content="Hello")` goes in through `agent.invoke({"messages": [message1]})`
2. **The graph passes it to your node**
   State now holds the conversation so far: ***[Hello]***
3. **chat_node reads it and calls the LLM**
   It reads `state["messages"]` and runs `llm.invoke(history)`
4. **The node returns only the NEW reply**
   `{"messages": response}` - an AIMessage like "Hi there!"
5. **add_messages merges it in**
   State becomes ***[Hello, Hi there!]*** - history kept, nothing replaced.

python

```python
def chat_node(state: AgentState) -> dict:
    conversation_history = state["messages"]
    response = llm.invoke(conversation_history)
    return {"messages": response}
```

reads the full history

returns just the new reply

Remember:

`List[BaseMessage]` only ***describes*** the data. It does not create the list, call the LLM or add messages. Your code and ***LangGraph*** do that.

Cheat sheet

everything on one page

Benefit	Why it matters
Conversation history	Keeps previous messages available to the LLM.
Automatic merging	You don't need to combine existing and returned messages by hand inside every node.
Cleaner code	Nodes can return only the state fields they want to update.
Message ID handling	Supports updating existing messages by ID.
Easier agent development	Useful when multiple nodes read or update the same conversation state.

Remember this

`List[BaseMessage]` = what the field stores

`Annotated` = attaches metadata to the type

`add_messages` = how LangGraph merges message updates

Annotated carries the note.

add_messages does the merging.